

**LAPORAN UJIAN TENGAH SEMESTER**  
***Implementasi Komunikasi Client-Server Amandengan***  
***Enkripsi AES-128-GCM pada Jaringan LAN***

**Mata Kuliah Network Programming & Administration**

**Kelas IFN41**



**Kelompok 5:**

**Marsani (230401010282), Muhammad Saifulloh (220401010207),  
Kristian Hananiel Hura (220401010289), Sukandar (240401020175)**

**Dosen Pengajar:**

**Abdul Azzam Ajhari, S.Kom., M.Kom**

**PROGRAM STUDI PJJ INFORMATIKA**  
**UNIVERSITAS SIBER ASIA**

**2026**

# LAPORAN UJIAN TENGAH SEMESTER

## Mata Kuliah: Network Programming & Administration

**Kelas** : IFN41  
**Program Studi** : Informatika PJJ S1  
**Kampus** : Universitas Siber Asia  
**Dosen Pengampu** : Abdul Azzam Ajhari, S.Kom., M.Kom.  
**Kelompok** : 5

### Anggota Kelompok dan Kontribusi

| No | Nama                   | NIM          | Kontribusi                                                                         |
|----|------------------------|--------------|------------------------------------------------------------------------------------|
| 1  | Marsani                | 230401010282 | Perancangan jaringan, diagram topologi, pemetaan OSI Layer, pengujian (Soal 1 & 4) |
| 2  | Muhammad Saifulloh     | 220401010207 | Implementasi program tanpa enkripsi, pengujian fungsional (Soal 2)                 |
| 3  | Kristian Hananiel Hura | 220401010289 | Analisis Wireshark, identifikasi kerentanan (Soal 3)                               |
| 4  | Sukandar               | 240401020175 | Implementasi AES-128-GCM, evaluasi komparatif (Soal 4 & 5)                         |

# Daftar Isi

|                                                                      |          |
|----------------------------------------------------------------------|----------|
| <b>LAPORAN UJIAN TENGAH SEMESTER.....</b>                            | <b>1</b> |
| Mata Kuliah: Network Programming & Administration.....               | 1        |
| Anggota Kelompok dan Kontribusi.....                                 | 1        |
| Daftar Isi.....                                                      | 2        |
| Pendahuluan.....                                                     | 4        |
| Soal 1: Analisis Kebutuhan dan Perancangan Jaringan.....             | 5        |
| 1.1 Perbedaan PAN, LAN, MAN, dan WAN.....                            | 5        |
| 1.2 Alasan Pemilihan LAN sebagai Lingkungan Implementasi.....        | 6        |
| 1.3 Diagram Topologi Jaringan.....                                   | 7        |
| 1.4 Alamat IP, Port, Protokol Transportasi, dan Arah Komunikasi..... | 8        |
| 1.5 Pemetaan Proses Komunikasi Terhadap Model OSI Layer.....         | 9        |
| 1.6 Diagram Alir Proses Pengiriman dan Penerimaan Pesan.....         | 11       |
| Soal 2: Implementasi Program Client–Server Tanpa Enkripsi.....       | 13       |
| 2.1 Kode Program Server (Tanpa Enkripsi).....                        | 13       |
| 2.2 Kode Program Client (Tanpa Enkripsi).....                        | 15       |
| 2.3 Penjelasan Fungsi Bagian-Bagian Utama Kode.....                  | 16       |
| 2.4 Risiko Keamanan pada Komunikasi Tanpa Enkripsi.....              | 18       |
| Soal 3: Analisis Kerentanan Menggunakan Wireshark.....               | 19       |
| 3.1 Tahapan Pengambilan Paket Komunikasi.....                        | 19       |
| 3.2 Filter Wireshark yang Digunakan.....                             | 20       |
| 3.3 Tangkapan Layar Paket yang Relevan.....                          | 21       |
| 3.4 Analisis Keterbacaan Isi Pesan.....                              | 22       |
| 3.5 Identifikasi Risiko Keamanan.....                                | 23       |
| 3.6 Solusi Keamanan yang Diperlukan.....                             | 24       |
| Soal 4: Implementasi Komunikasi Aman Menggunakan AES-128-GCM.....    | 25       |
| 4.1 Konsep Dasar Kriptografi yang Digunakan.....                     | 25       |

|                                                                      |    |
|----------------------------------------------------------------------|----|
| 4.2 Pengelolaan Kunci dalam Proyek.....                              | 27 |
| 4.3 Kode Program Server (Dengan Enkripsi AES-128-GCM).....           | 28 |
| 4.4 Kode Program Client (Dengan Enkripsi AES-128-GCM).....           | 30 |
| 4.5 Proses Enkripsi dan Dekripsi Secara Rinci.....                   | 32 |
| 4.6 Perlindungan Integritas pada AES-128-GCM.....                    | 33 |
| 4.7 Pengujian Ulang dengan Wireshark dan Perbandingan Hasil.....     | 34 |
| Soal 5: Evaluasi, Administrasi, dan Refleksi Proyek.....             | 35 |
| 5.1 Tabel Perbandingan Sebelum dan Sesudah Implementasi AES-128..... | 35 |
| 5.2 Perubahan Tingkat Keamanan Berdasarkan Bukti Wireshark.....      | 36 |
| 5.3 Risiko yang Masih Tersisa Setelah Enkripsi.....                  | 37 |
| 5.4 Rekomendasi Administrasi Keamanan Jaringan.....                  | 38 |
| 5.5 Jadwal Pengerjaan Proyek dan Pembagian Tugas.....                | 40 |
| 5.6 Refleksi Kelompok.....                                           | 41 |
| Kesimpulan.....                                                      | 43 |
| Daftar Pustaka.....                                                  | 44 |

# Pendahuluan

Sebagai Mahasiswa IT kami menyadari bahwa di era digital saat ini, pertukaran informasi melalui jaringan komputer sudah menjadi kebutuhan pokok bagi banyak organisasi, termasuk di lingkungan kampus dan lembaga pendidikan. Unit administrasi akademik misalnya, kerap mengirimkan pesan-pesan singkat berupa jadwal rapat, kode verifikasi internal, maupun status layanan kepada pihak-pihak terkait. Pesan-pesan semacam ini tentu saja menyimpan informasi yang bersifat sensitif dan tidak semestinya diketahui oleh pihak yang tidak berwenang.

Masalah muncul ketika komunikasi tersebut dilakukan tanpa adanya mekanisme pengamanan. Data yang dikirimkan dalam bentuk teks biasa (*plaintext*) melalui jaringan sangat rentan untuk disadap. Dengan bantuan perangkat lunak penganalisis jaringan seperti Wireshark, siapa pun yang berada di jaringan yang sama berpotensi membaca isi pesan secara utuh tanpa hambatan berarti.

Berdasarkan permasalahan tersebut, laporan ini menyajikan rancangan serta implementasi dua versi aplikasi komunikasi *client-server* berbasis Python. Versi pertama mengirimkan pesan tanpa enkripsi untuk membuktikan adanya kerentanan. Versi kedua memperbaiki kelemahan tersebut dengan menerapkan enkripsi AES-128 dalam mode GCM (*Galois/Counter Mode*), sehingga pesan yang melintas di jaringan tidak lagi dapat dibaca oleh pihak ketiga.

Melalui pengerjaan proyek ini, kelompok kami berharap dapat memahami secara praktis bagaimana prinsip-prinsip keamanan jaringan bekerja, sekaligus menyadari pentingnya penerapan kriptografi dalam setiap komunikasi data di lingkungan jaringan lokal maupun yang lebih luas.

# Soal 1: Analisis Kebutuhan dan Perancangan Jaringan

## 1.1 Perbedaan PAN, LAN, MAN, dan WAN

Untuk memahami konteks proyek ini, perlu kiranya kita mengenal empat klasifikasi jaringan berdasarkan cakupan geografisnya:

| Jenis Jaringan                                  | Cakupan          | Contoh Penerapan                                                                                                 | Karakteristik                                                                                                   |
|-------------------------------------------------|------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>PAN</b> ( <i>Personal Area Network</i> )     | 1–10 meter       | Koneksi <i>Bluetooth</i> antara ponsel dan <i>earphone</i> , koneksi <i>USB</i> antara laptop dan <i>printer</i> | Jangkauan sangat terbatas, digunakan untuk kebutuhan personal, kecepatan transfer bervariasi                    |
| <b>LAN</b> ( <i>Local Area Network</i> )        | 10 meter – 1 km  | Jaringan di dalam satu gedung kantor, laboratorium komputer kampus, jaringan rumah                               | Kecepatan tinggi (100 Mbps – 10 Gbps), latensi rendah, dikelola oleh satu organisasi                            |
| <b>MAN</b> ( <i>Metropolitan Area Network</i> ) | 1–50 km          | Jaringan antar-gedung dalam satu kota, jaringan kampus yang tersebar di beberapa lokasi                          | Menghubungkan beberapa LAN dalam satu wilayah metropolitan, sering menggunakan infrastruktur <i>fiber optic</i> |
| <b>WAN</b> ( <i>Wide Area Network</i> )         | Lebih dari 50 km | Internet, jaringan perusahaan multinasional yang menghubungkan kantor di berbagai negara                         | Cakupan paling luas, kecepatan relatif lebih rendah dibanding                                                   |

|  |  |  |                           |
|--|--|--|---------------------------|
|  |  |  | LAN, latensi lebih tinggi |
|--|--|--|---------------------------|

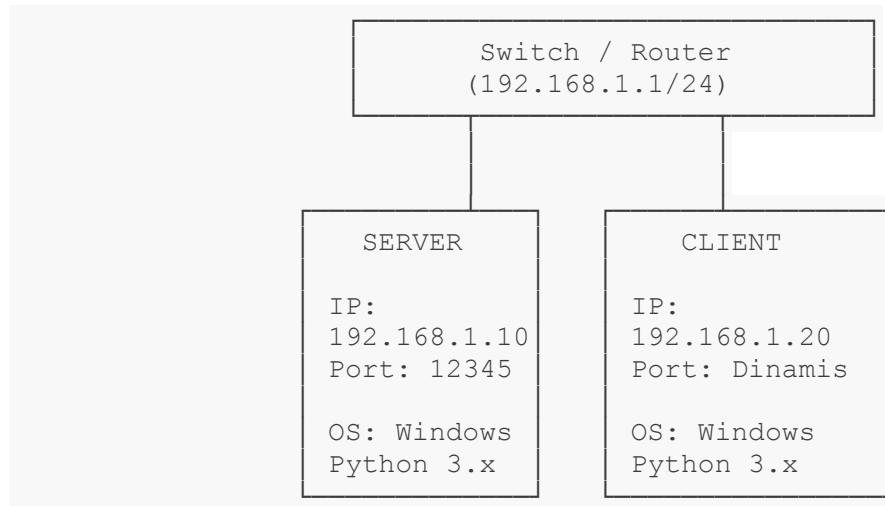
## 1.2 Alasan Pemilihan LAN sebagai Lingkungan Implementasi

Kelompok kami memilih LAN sebagai lingkungan implementasi proyek karena beberapa pertimbangan berikut:

1. **Kesesuaian dengan skenario nyata.** Komunikasi internal di unit administrasi akademik umumnya terjadi dalam satu gedung atau satu kompleks kampus. Jenis jaringan yang paling sesuai untuk kondisi tersebut adalah LAN, karena semua perangkat berada dalam jangkauan fisik yang relatif dekat.
2. **Kecepatan dan stabilitas yang memadai.** LAN menawarkan kecepatan transfer data yang tinggi dan latensi yang sangat rendah. Hal ini sangat mendukung komunikasi *real-time* antara *server* dan *client* sehingga pesan dapat dikirim dan diterima hampir seketika.
3. **Kemudahan dalam pengaturan dan pengujian.** Konfigurasi jaringan LAN relatif sederhana dibandingkan MAN atau WAN. Kami dapat dengan mudah menentukan alamat IP, melakukan pengujian koneksi, serta menangkap paket data menggunakan Wireshark tanpa membutuhkan infrastruktur yang rumit.
4. **Kontrol penuh terhadap jaringan.** Pada lingkungan LAN, kami memiliki kontrol penuh atas perangkat-perangkat yang terhubung. Ini memudahkan proses *debugging*, pemantauan lalu lintas data, serta penerapan kebijakan keamanan.
5. **Relevansi dengan materi perkuliahan.** Materi *Network Programming & Administration* berfokus pada pemrograman jaringan di lingkungan lokal. LAN menjadi landasan yang tepat sebelum mengembangkan pemahaman ke skala jaringan yang lebih besar.

### 1.3 Diagram Topologi Jaringan

Topologi yang digunakan dalam proyek ini adalah topologi *star* sederhana dengan satu *switch* sebagai pusat penghubung antara komputer *server* dan komputer *client*:



#### Keterangan Topologi:

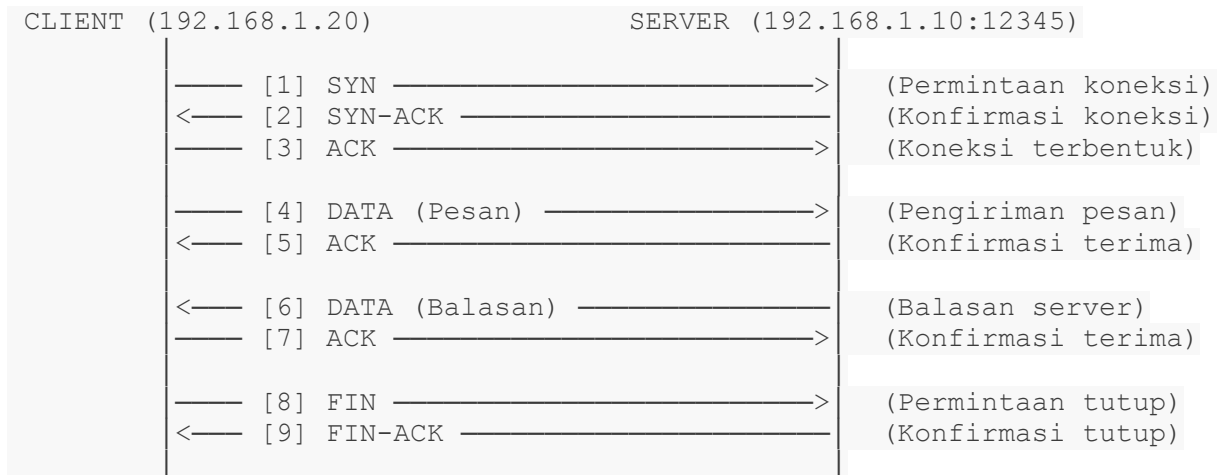
- *Server* ditempatkan pada alamat IP 192.168.1.10 dan mendengarkan koneksi pada port 12345.
- *Client* berada pada alamat IP 192.168.1.20 dan menggunakan port dinamis (*ephemeral port*) yang diberikan oleh sistem operasi secara otomatis.
- Kedua perangkat terhubung melalui *switch* dalam satu jaringan LAN dengan *subnet* 192.168.1.0/24.



## 1.4 Alamat IP, Port, Protokol Transportasi, dan Arah Komunikasi

| Parameter | Server                                       | Client                  |
|-----------|----------------------------------------------|-------------------------|
| Alamat IP | 192.168.1.10                                 | 192.168.1.20            |
| Port      | 12345 (tetap)                                | Dinamis (ditentukan OS) |
| Protokol  | TCP ( <i>Transmission Control Protocol</i> ) | TCP                     |
| Peran     | Menerima dan memproses pesan                 | Mengirim pesan          |

Arah komunikasi data:



Protokol TCP dipilih karena bersifat *connection-oriented*, yang berarti adanya jaminan bahwa data sampai secara utuh dan berurutan. Hal ini sesuai dengan kebutuhan komunikasi pesan dimana setiap karakter dalam pesan harus sampai dengan sempurna tanpa ada yang hilang atau tertukar.

## 1.5 Pemetaan Proses Komunikasi Terhadap Model OSI Layer

Setiap tahapan komunikasi dalam proyek ini dapat dipetakan ke tujuh lapisan model OSI (*Open Systems Interconnection*) sebagai berikut:

| Layer | Nama Layer          | Fungsi dalam Proyek                                                                                                                                                                              | Komponen yang Berperan                                                                                                       |
|-------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| 7     | <b>Application</b>  | Pengguna memasukkan pesan melalui program Python. Program memproses input dan menyiapkan data untuk dikirim.                                                                                     | Program Python ( <code>client.py</code> , <code>server.py</code> ), fungsi <code>input()</code> , <code>print()</code>       |
| 6     | <b>Presentation</b> | Pesan diubah dari teks menjadi format byte menggunakan <i>encoding</i> UTF-8. Pada versi terenkripsi, pesan diubah menjadi <i>ciphertext</i> dengan AES-128-GCM lalu dikodekan ke format Base64. | Fungsi <code>encode('utf-8')</code> , <code>decode('utf-8')</code> , library <code>cryptography</code> , <code>base64</code> |
| 5     | <b>Session</b>      | Sesi koneksi TCP dibuka antara <i>client</i> dan <i>server</i> . Sesi ini dikelola selama proses pengiriman dan penerimaan pesan berlangsung, lalu ditutup setelah selesai.                      | <code>socket.connect()</code> , <code>socket.accept()</code> , <code>socket.close()</code>                                   |

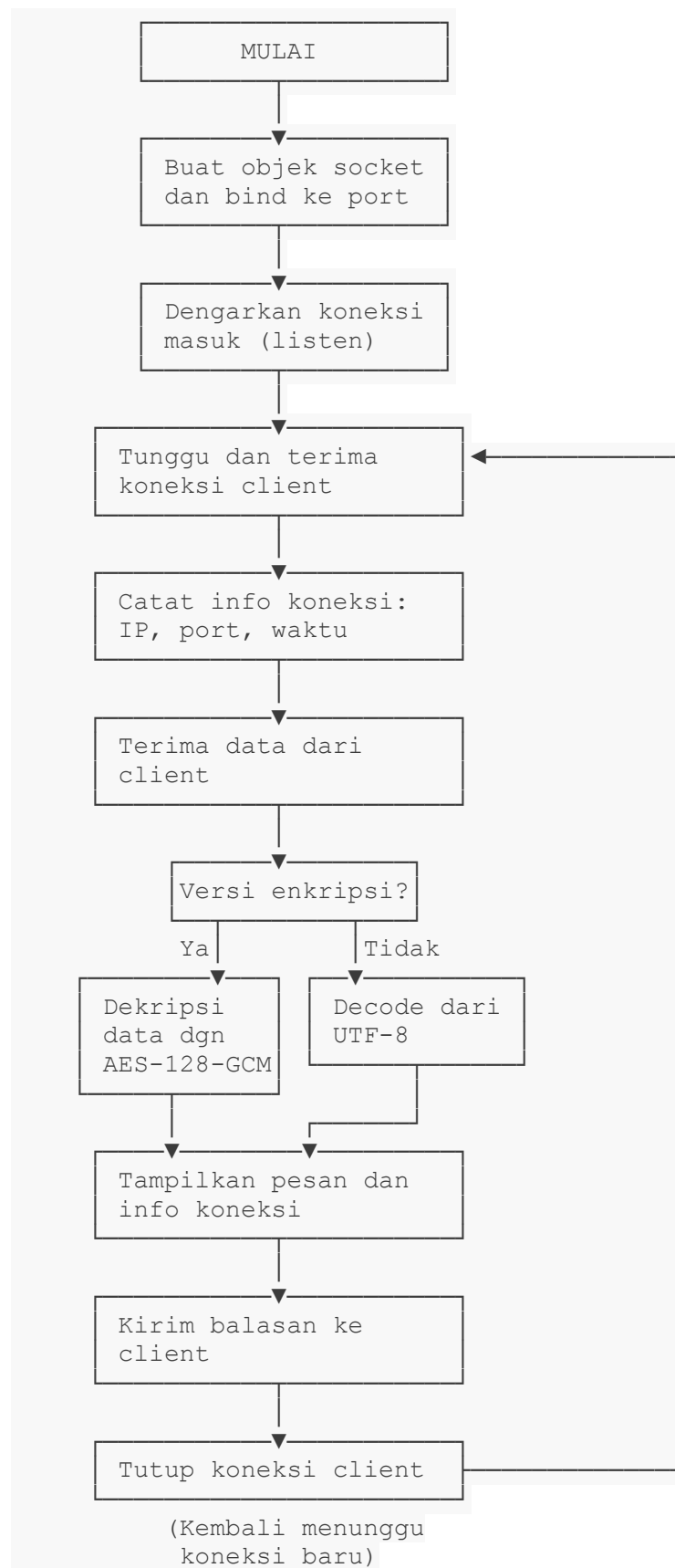
|   |                  |                                                                                                                                                                    |                                                                                      |
|---|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| 4 | <b>Transport</b> | Data dipecah menjadi segmen-segmen TCP. Protokol TCP menjamin pengiriman data secara andal ( <i>reliable</i> ), berurutan, dan tanpa duplikasi.                    | TCP ( <i>Transmission Control Protocol</i> ), port 12345, <i>three-way handshake</i> |
| 3 | <b>Network</b>   | Setiap segmen dibungkus dalam paket IP yang berisi alamat IP sumber dan tujuan. Paket diarahkan ( <i>routed</i> ) menuju perangkat tujuan.                         | IPv4, alamat IP 192.168.1.10 dan 192.168.1.20                                        |
| 2 | <b>Data Link</b> | Paket IP dienkapsulasi menjadi <i>frame</i> Ethernet yang dilengkapi alamat MAC sumber dan tujuan. <i>Frame</i> ini dikirim melalui media fisik di jaringan LAN.   | Ethernet, alamat MAC, <i>switch</i>                                                  |
| 1 | <b>Physical</b>  | <i>Frame</i> Ethernet dikonversi menjadi sinyal listrik atau cahaya untuk ditransmisikan melalui kabel jaringan ( <i>UTP Cat 5e/6</i> ) atau sinyal radio (Wi-Fi). | Kabel UTP / Wi-Fi, NIC ( <i>Network Interface Card</i> )                             |

## 1.6 Diagram Alir Proses Pengiriman dan Penerimaan Pesan

Diagram alir sisi client (pengirim):



Diagram alir sisi server (penerima):



## Soal 2: Implementasi Program Client–Server Tanpa Enkripsi

### 2.1 Kode Program Server (Tanpa Enkripsi)

Kode program *server* disimpan dalam file `server_tanpa_enkripsi.py`. Berikut adalah kode beserta penjelasan fungsi bagian-bagian utamanya:

```
import socket
from datetime import datetime

HOST = '0.0.0.0'
PORT = 12345

def jalankan_server():
    # Membuat objek socket TCP/IPv4
    server_socket = socket.socket(socket.AF_INET,
                                   socket.SOCK_STREAM)

    # Mengatur agar port dapat langsung digunakan kembali
    server_socket.setsockopt(socket.SOL_SOCKET,
                              socket.SO_REUSEADDR, 1)

    # Mengikat socket ke alamat dan port
    server_socket.bind((HOST, PORT))

    # Mulai mendengarkan koneksi (antrian maksimal 5)
    server_socket.listen(5)

    print(f"[SERVER] Berjalan di {HOST}:{PORT}")
    print(f"[SERVER] Menunggu koneksi dari client...\n")

    try:
        while True:
            # Menerima koneksi dari client
            client_socket, alamat_client =
server_socket.accept()
            ip_client = alamat_client[0]
            port_client = alamat_client[1]
            waktu_koneksi = datetime.now().strftime("%Y-%m-%d
%H:%M:%S")
```

```

        print(f"[KONEKSI BARU] Client terhubung")
        print(f"    Alamat IP    : {ip_client}")
        print(f"    Port          : {port_client}")
        print(f"    Waktu         : {waktu_koneksi}")

    # Menerima data dari client
    data = client_socket.recv(4096)

    if data:
        pesan = data.decode('utf-8')
        waktu_terima = datetime.now().strftime("%Y-%m-%d
%H:%M:%S")

        print(f"\n[PESAN DITERIMA]")
        print(f"    Dari          :
{ip_client}:{port_client}")
        print(f"    Waktu         : {waktu_terima}")
        print(f"    Isi Pesan    : {pesan}")

        balasan = f"Pesan diterima oleh server pada
{waktu_terima}"
        client_socket.send(balasan.encode('utf-8'))

    client_socket.close()

except KeyboardInterrupt:
    print("\n[SERVER] Dihentikan oleh pengguna.")
finally:
    server_socket.close()

if __name__ == "__main__":
    jalankan_server()

```

## 2.2 Kode Program Client (Tanpa Enkripsi)

Kode program *client* disimpan dalam file `client_tanpa_enkripsi.py`:

```
import socket

SERVER_HOST = '127.0.0.1'
SERVER_PORT = 12345

def jalankan_client():
    client_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)

    try:
        client_socket.connect((SERVER_HOST, SERVER_PORT))
        print(f"[CLIENT] Terhubung ke server
{SERVER_HOST}:{SERVER_PORT}")

        pesan = input("\nMasukkan pesan yang ingin dikirim: ")

        client_socket.send(pesan.encode('utf-8'))
        print(f"[CLIENT] Pesan terkirim: {pesan}")

        balasan = client_socket.recv(4096).decode('utf-8')
        print(f"[CLIENT] Balasan server: {balasan}")

    except ConnectionRefusedError:
        print("[ERROR] Tidak dapat terhubung ke server.")
    except Exception as e:
        print(f"[ERROR] Terjadi kesalahan: {e}")
    finally:
        client_socket.close()
        print("[CLIENT] Koneksi ditutup.")

if __name__ == "__main__":
    jalankan_client()
```



## 2.3 Penjelasan Fungsi Bagian-Bagian Utama Kode

| Bagian Kode                                      | Fungsi                           | Penjelasan                                                                                                                                                                                        |
|--------------------------------------------------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>socket.socket(AF_INET, SOCK_STREAM)</code> | Membuat objek <i>socket</i>      | <code>AF_INET</code> menandakan penggunaan protokol IPv4, sedangkan <code>SOCK_STREAM</code> menandakan penggunaan TCP yang menjamin pengiriman data secara andal dan berurutan.                  |
| <code>server_socket.bind((HOST, PORT))</code>    | Mengikat <i>socket</i> ke alamat | <i>Server</i> mendaftarkan dirinya pada alamat IP dan port tertentu sehingga dapat ditemukan oleh <i>client</i> .<br><code>0.0.0.0</code> berarti menerima koneksi dari semua antarmuka jaringan. |
| <code>server_socket.listen(5)</code>             | Mendengarkan koneksi             | <i>Server</i> mulai mendengarkan koneksi masuk. Angka 5 menunjukkan jumlah maksimal koneksi yang dapat mengantri sebelum diproses.                                                                |
| <code>server_socket.accept()</code>              | Menerima koneksi                 | Menunggu koneksi dari <i>client</i> lalu mengembalikan objek <i>socket</i> baru untuk berkomunikasi dengan <i>client</i> tersebut, beserta alamat IP dan port-nya.                                |
| <code>client_socket.connect((HOST, PORT))</code> | Menghubungkan ke <i>server</i>   | <i>Client</i> memulai proses <i>three-way handshake</i> TCP untuk membangun koneksi dengan <i>server</i> pada alamat dan port yang ditentukan.                                                    |

|                                           |                |                                                                                                                                                                                                       |
|-------------------------------------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>client_socket.send(data)</code>     | Mengirim data  | Mengirimkan data dalam bentuk <i>byte</i> melalui koneksi TCP. Pesan teks perlu dikonversi terlebih dahulu menggunakan <code>.encode('utf-8')</code> .                                                |
| <code>client_socket.recv(4096)</code>     | Menerima data  | Menerima data dari koneksi TCP dengan ukuran <i>buffer</i> maksimal 4096 <i>byte</i> . Data yang diterima berupa <i>byte</i> dan perlu dikonversi kembali menggunakan <code>.decode('utf-8')</code> . |
| <code>datetime.now().strftime(...)</code> | Mencatat waktu | Mengambil waktu saat ini dan memformatnya menjadi string yang mudah dibaca untuk keperluan pencatatan ( <i>logging</i> ).                                                                             |

## 2.4 Risiko Keamanan pada Komunikasi Tanpa Enkripsi

Pesan yang dikirimkan tanpa perlindungan kriptografi menyimpan risiko keamanan yang serius. Berikut alasannya:

1. **Pesan dikirim dalam bentuk teks terbuka (*plaintext*).** Ketika *client* memanggil `pesan.encode('utf-8')`, data yang dikirim melalui jaringan hanyalah representasi *byte* dari teks asli tanpa perubahan apapun. Ini berarti siapa pun yang berhasil menangkap paket data tersebut dapat langsung membaca isi pesan tanpa perlu kunci atau proses dekripsi.
2. **Tidak ada mekanisme verifikasi integritas.** Tidak ada cara untuk memastikan apakah pesan yang diterima *server* benar-benar sama dengan pesan yang dikirim *client*. Pesan bisa saja dimodifikasi di tengah perjalanan (*man-in-the-middle attack*) tanpa ada pihak yang menyadarinya.
3. **Tidak ada mekanisme autentikasi.** *Server* tidak dapat memastikan bahwa pesan benar-benar berasal dari *client* yang sah. Pihak manapun yang mengetahui alamat IP dan port *server* dapat mengirim pesan palsu.
4. **Perangkat lunak penganalisis jaringan mudah didapat.** Alat seperti Wireshark bersifat *open-source* dan gratis. Siapa pun dalam jaringan LAN yang sama dapat menjalankannya untuk menangkap dan membaca seluruh lalu lintas data tanpa memerlukan keahlian teknis yang tinggi.

Hasil pengujian pada Soal 3 akan membuktikan secara konkret betapa mudahnya pesan tanpa enkripsi ini dibaca oleh pihak ketiga.

## Soal 3: Analisis Kerentanan Menggunakan Wireshark

### 3.1 Tahapan Pengambilan Paket Komunikasi

Proses pengambilan paket (*packet capture*) dilakukan melalui tahapan-tahapan berikut:

1. **Persiapan lingkungan pengujian.** Pastikan komputer *server* dan *client* terhubung dalam satu jaringan LAN. Catat alamat IP masing-masing perangkat.
2. **Membuka Wireshark.** Jalankan Wireshark pada salah satu komputer (bisa di *server*, *client*, atau komputer ketiga yang terhubung di jaringan yang sama).
3. **Memilih antarmuka jaringan.** Pilih antarmuka jaringan yang digunakan untuk komunikasi. Jika menggunakan kabel, pilih antarmuka Ethernet. Jika menggunakan Wi-Fi, pilih antarmuka *wireless*. Untuk pengujian di satu komputer (*localhost*), pilih antarmuka *Loopback* atau *Adapter for Loopback Traffic Capture* (Npcap).
4. **Memulai penangkapan paket.** Klik tombol *Start Capturing* (ikon sirip hiu berwarna biru) untuk mulai merekam seluruh lalu lintas jaringan.
5. **Menjalankan aplikasi.** Jalankan `server_tanpa_enkripsi.py` pada terminal pertama, lalu jalankan `client_tanpa_enkripsi.py` pada terminal kedua. Kirimkan pesan pengujian, misalnya: Rapat dosen pukul 10:00 di Ruang Utama.
6. **Menghentikan penangkapan.** Setelah pesan berhasil terkirim dan balasan diterima, klik tombol *Stop Capturing* (ikon kotak merah) untuk menghentikan perekaman.
7. **Menyimpan hasil tangkapan.** Simpan file hasil tangkapan dengan format `.pcapng` untuk dokumentasi dan analisis lebih lanjut.

### 3.2 Filter Wireshark yang Digunakan

Untuk menyaring paket-paket yang relevan dengan komunikasi *client-server* kita, beberapa filter berikut dapat digunakan:

| Filter                                                            | Kegunaan                                                                               |
|-------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <code>tcp.port == 12345</code>                                    | Menampilkan semua paket TCP yang menggunakan port 12345 ( <i>portserver</i> kita)      |
| <code>ip.addr == 192.168.1.10</code>                              | Menampilkan semua paket yang melibatkan alamat <i>IPserver</i>                         |
| <code>ip.addr == 192.168.1.20 &amp;&amp; tcp.port == 12345</code> | Menampilkan paket dari/ <i>keclient</i> yang menuju port <i>server</i>                 |
| <code>tcp.port == 12345 &amp;&amp; data</code>                    | Menampilkan hanya paket data (bukan paket kontrol TCP seperti SYN/ACK) pada port 12345 |
| <code>tcp.stream eq 0</code>                                      | Menampilkan seluruh paket dalam satu sesi koneksi TCP tertentu                         |

Apabila pengujian dilakukan di satu komputer menggunakan *localhost*, gunakan filter:

- `tcp.port == 12345` (paling sederhana dan efektif)
- `ip.addr == 127.0.0.1 && tcp.port == 12345`

### 3.3 Tangkapan Layar Paket yang Relevan

**Catatan:** Tangkapan layar akan diambil secara manual saat program dijalankan. Berikut adalah deskripsi tiga tangkapan layar yang perlu didokumentasikan.

#### Tangkapan Layar 1: Proses *Three-Way Handshake*

Tangkapan layar ini menunjukkan tiga paket awal yang membentuk koneksi TCP antara *client* dan *server*:

- Paket SYN dari *client* ke *server* (permintaan koneksi)
- Paket SYN-ACK dari *server* ke *client* (konfirmasi kesediaan)
- Paket ACK dari *client* ke *server* (koneksi resmi terbentuk)

Pada kolom *Info* di Wireshark, kita akan melihat urutan: [SYN] → [SYN, ACK] → [ACK].

#### Tangkapan Layar 2: Paket Data Berisi Pesan (*Plaintext*)

Ini adalah tangkapan layar paling krusial. Ketika kita memilih paket data yang dikirim *client* dan melihat bagian *payload* di panel bawah Wireshark (bagian *hexdump*), isi pesan akan terlihat jelas dalam bentuk teks yang dapat dibaca. Misalnya, jika pesan yang dikirim adalah Rapat dosen pukul 10:00 di Ruang Utama, maka teks tersebut akan muncul secara utuh di bagian data paket.

Caranya: klik kanan pada paket data → pilih *Follow* → *TCP Stream*. Jendela baru akan menampilkan seluruh percakapan dalam bentuk teks biasa.

#### Tangkapan Layar 3: Hasil *Follow TCP Stream*

Fitur *Follow TCP Stream* pada Wireshark akan menampilkan seluruh data yang dipertukarkan dalam satu sesi koneksi TCP. Pada tangkapan layar ini, akan terlihat:

- Pesan yang dikirim *client* (biasanya ditampilkan dengan warna merah)
- Balasan yang dikirim *server* (biasanya ditampilkan dengan warna biru)

Seluruh isi percakapan terbaca dengan jelas, membuktikan bahwa komunikasi tanpa enkripsi sangat rentan terhadap penyadapan.

### 3.4 Analisis Keterbacaan Isi Pesan

Berdasarkan hasil tangkapan Wireshark, isi pesan **dapat terbaca secara utuh dan jelas** tanpa memerlukan proses dekripsi apapun. Ketika kami menggunakan fitur *Follow TCP Stream*, seluruh percakapan antara *client* dan *server* ditampilkan dalam bentuk teks biasa. Ini membuktikan bahwa:

- Data dikirim dalam bentuk *plaintext* tanpa pengacakan.
- Tidak ada lapisan perlindungan antara data asli dan data yang melintas di jaringan.
- Siapa pun dengan akses ke jaringan dan perangkat lunak *packet sniffer* dapat membaca pesan tersebut.

### 3.5 Identifikasi Risiko Keamanan

Berdasarkan analisis di atas, berikut adalah lima risiko keamanan yang berhasil diidentifikasi:

#### 1. Penyadapan Pesan (*Eavesdropping / Sniffing*)

Risiko paling mendasar adalah kemampuan pihak ketiga untuk membaca isi pesan secara langsung. Dalam jaringan LAN, terutama yang menggunakan Wi-Fi atau *hub*, semua perangkat yang terhubung berpotensi menerima paket data yang seharusnya bukan ditujukan untuk mereka. Dengan Wireshark, penyerang cukup menjalankan *packet capture* untuk membaca seluruh komunikasi.

#### 2. Kebocoran Informasi (*Information Disclosure*)

Pesan-pesan yang berisi informasi sensitif seperti jadwal rapat internal, kode verifikasi, atau status layanan akademik dapat bocor ke pihak yang tidak berwenang. Kebocoran ini dapat disalahgunakan untuk kepentingan yang merugikan institusi.

#### 3. Manipulasi Pesan (*Message Tampering / Man-in-the-Middle*)

Karena tidak ada mekanisme untuk memeriksa keutuhan (*integrity*) pesan, seorang penyerang yang berhasil mencegat komunikasi dapat memodifikasi isi pesan sebelum meneruskannya ke tujuan. *Server* tidak memiliki cara untuk membedakan pesan asli dengan pesan yang sudah dimanipulasi.

#### 4. Impersonasi *Client* (*Client Spoofing*)

Tanpa mekanisme autentikasi, siapa pun dapat membuat program *client* dan mengirim pesan ke *server* dengan berpura-pura menjadi pengguna yang sah. *Server* akan memproses pesan tersebut tanpa curiga karena tidak ada proses verifikasi identitas pengirim.

#### 5. Serangan *Replay*

Penyerang dapat menangkap paket data yang sah menggunakan Wireshark, lalu mengirim ulang paket tersebut di kemudian hari. Karena tidak ada mekanisme pembeda antara pesan baru dan



pesan lama (seperti *timestamp* atau *nonce*), *server* akan memproses pesan yang dikirim ulang tersebut seolah-olah itu adalah pesan baru yang valid.

### 3.6 Solusi Keamanan yang Diperlukan

Untuk mengatasi risiko-risiko di atas, beberapa solusi keamanan berikut perlu diterapkan:

1. **Enkripsi data** menggunakan algoritma kriptografi yang kuat seperti AES-128 agar isi pesan tidak dapat dibaca meskipun paket data berhasil ditangkap.
2. **Authenticated encryption** (misalnya AES-GCM) yang tidak hanya mengenkripsi pesan tetapi juga menyertakan *authentication tag* untuk memverifikasi bahwa pesan tidak dimodifikasi selama pengiriman.
3. **Pertukaran kunci yang aman** menggunakan protokol seperti Diffie-Hellman atau TLS *handshake* untuk memastikan kunci enkripsi tidak perlu dikirim secara terbuka melalui jaringan.
4. **Penggunaan *nonce* atau *timestamp*** dalam setiap pesan untuk mencegah serangan *replay*. Setiap pesan harus memiliki penanda unik yang membuatnya tidak dapat digunakan kembali.
5. **Autentikasi *client*** melalui mekanisme seperti sertifikat digital, *token*, atau kredensial untuk memastikan bahwa pesan benar-benar berasal dari pengguna yang sah.

## Soal 4: Implementasi Komunikasi Aman Menggunakan AES-128-GCM

### 4.1 Konsep Dasar Kriptografi yang Digunakan

Sebelum membahas implementasi, perlu dipahami beberapa istilah dasar dalam kriptografi:

| Istilah                                   | Penjelasan                                                                                                                                                          | Contoh dalam Proyek                                                                  |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>Plaintext</b>                          | Pesan asli yang masih dalam bentuk teks biasa dan dapat dibaca oleh siapa saja.                                                                                     | Rapat dosen pukul 10:00 di Ruang Utama                                               |
| <b>Ciphertext</b>                         | Pesan yang sudah diacak melalui proses enkripsi sehingga tidak dapat dibaca tanpa kunci yang benar.                                                                 | aG9sZXRoaxNpc2Vu... (karakter acak dalam Base64)                                     |
| <b>Key (Kunci)</b>                        | Nilai rahasia yang digunakan untuk mengenkripsi dan mendekripsi pesan. Panjang kunci menentukan tingkat keamanan. Dalam AES-128, kunci sepanjang 128 bit (16 byte). | Disimpan dalam file kunci_rahasia.key                                                |
| <b>Nonce / IV (Initialization Vector)</b> | Nilai acak yang digunakan bersama kunci untuk memastikan bahwa pesan yang sama menghasilkan <i>ciphertext</i> yang berbeda setiap kali                              | Dibuat secara acak menggunakan <code>os.urandom(12)</code> setiap kali pesan dikirim |

|  |                                                    |  |
|--|----------------------------------------------------|--|
|  | dienkripsi. Pada AES-GCM, nonce sepanjang 12 byte. |  |
|--|----------------------------------------------------|--|

### Mengapa menggunakan AES-128-GCM?

AES-GCM (*Galois/Counter Mode*) adalah mode operasi yang termasuk dalam kategori *authenticated encryption*. Keunggulannya dibandingkan mode AES lain (seperti ECB atau CBC) adalah:

- **Kerahasiaan (*confidentiality*)**: Pesan dienkripsi sehingga tidak dapat dibaca oleh pihak yang tidak memiliki kunci.
- **Integritas (*integrity*)**: GCM menghasilkan *authentication tag* yang memungkinkan penerima memverifikasi bahwa pesan tidak diubah selama pengiriman.
- **Autentikasi (*authentication*)**: *Tag* juga memastikan bahwa pesan benar-benar dienkripsi oleh pihak yang memiliki kunci yang sah.

Dengan kata lain, AES-GCM mengatasi tiga dari lima risiko yang diidentifikasi pada Soal 3 sekaligus (penyadapan, manipulasi pesan, dan sebagian impersonasi).

## 4.2 Pengelolaan Kunci dalam Proyek

Pengelolaan kunci (*key management*) merupakan aspek kritis dalam implementasi kriptografi. Dalam proyek ini, kami menerapkan strategi pengelolaan kunci sebagai berikut:

1. **Pembuatan kunci secara otomatis.** Saat *server* pertama kali dijalankan dan belum ada file kunci, program akan membuat kunci AES-128 baru secara acak menggunakan fungsi `AESGCM.generate_key(bit_length=128)` dari library `cryptography`. Fungsi ini menggunakan *cryptographically secure random number generator* yang disediakan oleh sistem operasi.
2. **Penyimpanan kunci dalam file terpisah.** Kunci disimpan dalam file `kunci_rahasia.key` yang terpisah dari *source code*. Pendekatan ini lebih aman dibandingkan menuliskan kunci langsung di dalam kode program karena:
  - File kunci dapat diberikan izin akses (*permission*) yang lebih ketat.
  - Kunci tidak ikut tersebar jika *source code* dibagikan atau diunggah ke *repository*.
  - Kunci dapat diganti tanpa perlu mengubah kode program.
3. **Distribusi kunci secara manual.** File kunci harus disalin secara manual dari komputer *server* ke komputer *client* melalui media yang aman, misalnya *flash drive* atau transfer file terenkripsi. Kunci tidak pernah dikirim melalui jaringan dalam proyek ini.
4. **Dukungan *environment variable*.** Sebagai alternatif, kunci dapat disimpan dalam *environment variable* (`AES_KEY`) sehingga tidak perlu berupa file fisik di *disk*.

### 4.3 Kode Program Server (Dengan Enkripsi AES-128-GCM)

```
import socket
from datetime import datetime
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import base64
import os

HOST = '0.0.0.0'
PORT = 12345

# Memuat kunci dari file
KUNCI_FILE = 'kunci_rahasia.key'
if os.path.exists(KUNCI_FILE):
    with open(KUNCI_FILE, 'rb') as f:
        KUNCI_RAHASIA = f.read()
else:
    KUNCI_RAHASIA = AESGCM.generate_key(bit_length=128)
    with open(KUNCI_FILE, 'wb') as f:
        f.write(KUNCI_RAHASIA)

def dekripsi_pesan(data_terenkripsi):
    """
    Mendekripsi pesan menggunakan AES-128-GCM.
    Data masukan berformat base64 yang berisi nonce + ciphertext
    + tag.
    """
    data_mentah = base64.b64decode(data_terenkripsi)
    nonce = data_mentah[:12] # 12 byte pertama = nonce
    ciphertext_dan_tag = data_mentah[12:] # Sisanya =
    ciphertext + tag

    aesgcm = AESGCM(KUNCI_RAHASIA)
    plaintext = aesgcm.decrypt(nonce, ciphertext_dan_tag, None)
    return plaintext.decode('utf-8')

def jalankan_server():
    server_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    server_socket.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEADDR, 1)
    server_socket.bind((HOST, PORT))
    server_socket.listen(5)

    print(f"[SERVER AMAN] Mode: AES-128-GCM")
    print(f"[SERVER AMAN] Menunggu koneksi...\n")
```

```

try:
    while True:
        client_socket, alamat = server_socket.accept()
        ip, port = alamat
        waktu = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        print(f"[KONEKSI] {ip}:{port} pada {waktu}")

        data = client_socket.recv(4096)
        if data:
            print(f"[CIPHERTEXT] {data.decode('utf-8')[:60]}...")
            try:
                pesan = dekripsi_pesan(data)
                print(f"[PLAINTEXT] {pesan}\n{'-'*50}")
                client_socket.send(b"Pesan diterima dan didekripsi.")
            except Exception as e:
                print(f"[ERROR] Dekripsi gagal: {e}")
                client_socket.send(b"ERROR: Dekripsi gagal")

        client_socket.close()
    except KeyboardInterrupt:
        print("\n[SERVER] Dihentikan.")
    finally:
        server_socket.close()

if __name__ == "__main__":
    jalankan_server()

```

## 4.4 Kode Program Client (Dengan Enkripsi AES-128-GCM)

```
import socket
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import base64
import os

SERVER_HOST = '127.0.0.1'
SERVER_PORT = 12345
KUNCI_FILE = 'kunci_rahasia.key'

def muat_kunci():
    """Memuat kunci AES-128 dari file."""
    with open(KUNCI_FILE, 'rb') as f:
        return f.read()

def enkripsi_pesan(pesan, kunci):
    """
    Mengenkripsi pesan menggunakan AES-128-GCM.
    Mengembalikan data base64 berisi nonce + ciphertext + tag.
    """
    nonce = os.urandom(12) # Nonce acak 12 byte
    aesgcm = AESGCM(kunci)
    ciphertext_dan_tag = aesgcm.encrypt(nonce,
    pesan.encode('utf-8'), None)
    return base64.b64encode(nonce + ciphertext_dan_tag)

def jalankan_client():
    kunci = muat_kunci()
    client_socket = socket.socket(socket.AF_INET,
    socket.SOCK_STREAM)

    try:
        client_socket.connect((SERVER_HOST, SERVER_PORT))
        print(f"[CLIENT AMAN] Terhubung, mode AES-128-GCM")

        pesan = input("\nMasukkan pesan: ")
        data_terenkripsi = enkripsi_pesan(pesan, kunci)

        print(f"[PLAINTEXT] {pesan}")
        print(f"[CIPHERTEXT]
{data_terenkripsi.decode()[:60]}...")

        client_socket.send(data_terenkripsi)
        balasan = client_socket.recv(4096).decode('utf-8')
        print(f"[SERVER] {balasan}")
```

```
except ConnectionRefusedError:
    print("[ERROR] Server tidak tersedia.")
finally:
    client_socket.close()

if __name__ == "__main__":
    jalankan_client()
```



## 4.5 Proses Enkripsi dan Dekripsi Secara Rinci

Untuk memberikan gambaran yang lebih jelas, berikut adalah alur lengkap proses enkripsi dan dekripsi:

### PROSES ENKRIPSI (Sisi Client):

```
Pesan Asli : "Rapat dosen pukul 10:00"
↓
[1] Buat nonce acak 12 byte
    |   nonce = os.urandom(12)
    |   Contoh: b'\x9a\x3b\xfa...'
    ↓
[2] Enkripsi dengan AES-128-GCM
    |   Input: plaintext + kunci + nonce
    |   Output: ciphertext + authentication tag (16 byte)
    ↓
[3] Gabungkan: nonce + ciphertext + tag
    |   Total = 12 + len(pesan) + 16 byte
    ↓
[4] Encode ke Base64 untuk pengiriman via TCP
    |   Output: "mjvy8kL2n5..."
    ↓
Data siap dikirim ke server
```

### PROSES DEKRIPSI (Sisi Server):

```
Data diterima: "mjvy8kL2n5..."
↓
[1] Decode dari Base64
    |   Output: byte mentah (nonce + ciphertext + tag)
    ↓
[2] Pisahkan nonce (12 byte pertama) dari ciphertext+tag
    |   nonce = data[:12]
    |   ciphertext_tag = data[12:]
    ↓
[3] Dekripsi dengan AES-128-GCM
    |   Input: ciphertext + tag + kunci + nonce
    |   Proses: verifikasi tag → dekripsi
    |   Jika tag tidak cocok → ERROR (pesan telah dimanipulasi)
    ↓
[4] Decode byte ke string UTF-8
```

```
| Output: "Rapat dosen pukul 10:00"
▼
Pesan asli ditampilkan
```

---

## 4.6 Perlindungan Integritas pada AES-128-GCM

AES-GCM secara otomatis menyediakan perlindungan integritas melalui mekanisme *authentication tag*. Berikut cara kerjanya:

1. Selama proses enkripsi, GCM tidak hanya menghasilkan *ciphertext* tetapi juga menghitung sebuah *tag* autentikasi sepanjang 16 byte. *Tag* ini merupakan semacam "sidik jari" dari kombinasi *plaintext*, kunci, dan *nonce*.
2. Saat dekripsi, GCM menghitung ulang *tag* dari *ciphertext* yang diterima dan membandingkannya dengan *tag* yang dikirim bersama data. Jika kedua *tag* tidak cocok — yang berarti pesan telah dimodifikasi — proses dekripsi akan gagal dan menghasilkan *exception*.
3. Dalam kode kami, jika seseorang memodifikasi *ciphertext* selama pengiriman, fungsi `aesgcm.decrypt()` akan melempar *exception* `InvalidTag`, dan pesan tidak akan ditampilkan. Ini jauh lebih aman dibandingkan mode AES-CBC yang memerlukan penambahan HMAC secara terpisah untuk verifikasi integritas.

## 4.7 Pengujian Ulang dengan Wireshark dan Perbandingan Hasil

Setelah menjalankan versi terenkripsi dan menangkap paketnya menggunakan Wireshark, perbedaan yang sangat jelas terlihat dibandingkan dengan hasil pada Soal 3:

| Aspek             | Tanpa Enkripsi (Soal 3)                      | Dengan AES-128-GCM (Soal 4)                                                                            |
|-------------------|----------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Isi paket data    | Pesan terbaca jelas sebagai teks biasa       | Data berupa karakter acak (Base64 <i>dariciphertext</i> ) yang tidak bermakna                          |
| Follow TCP Stream | Seluruh percakapan terbaca                   | Hanya terlihat deretan karakter acak yang tidak dapat dipahami                                         |
| Hexdump           | Karakter ASCII pesan terlihat di kolom kanan | Tidak ada pola karakter yang bermakna                                                                  |
| Ukuran data       | Sama persis dengan panjang pesan asli        | Lebih besar karena ada tambah <i>nonce</i> (12 byte), <i>tag</i> (16 byte), dan <i>overhead</i> Base64 |

Dengan kata lain, meskipun paket data tetap dapat ditangkap oleh Wireshark, isi pesan sudah tidak dapat dibaca. Penyerang hanya akan melihat data acak yang tidak berguna tanpa memiliki kunci dekripsi yang benar.

## Soal 5: Evaluasi, Administrasi, dan Refleksi Proyek

### 5.1 Tabel Perbandingan Sebelum dan Sesudah Implementasi AES-128

| Kriteria                                   | Versi Tanpa Enkripsi                                                | Versi Dengan AES-128-GCM                                                                                                       |
|--------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Kerahasiaan pesan</b>                   | Tidak ada — pesan dapat dibaca oleh siapa saja yang menangkap paket | Terjamin — pesan dienkripsi menjadi <i>ciphertext</i> yang tidak bermakna tanpa kunci                                          |
| <b>Integritas pesan</b>                    | Tidak ada — pesan dapat dimodifikasi tanpa terdeteksi               | Terjamin — <i>authentication tag</i> mendeteksi setiap perubahan pada <i>ciphertext</i>                                        |
| <b>Autentikasi pengirim</b>                | Tidak ada — siapa pun dapat mengirim pesan ke <i>server</i>         | Parsial — hanya pihak yang memiliki kunci yang dapat membuat <i>ciphertext</i> valid                                           |
| <b>Keterbacaan di Wireshark</b>            | Pesan terbaca jelas dalam <i>plaintext</i>                          | Hanya terlihat data acak ( <i>ciphertext</i> dalam format Base64)                                                              |
| <b>Perlindungan terhadap <i>replay</i></b> | Tidak ada                                                           | Parsial — <i>nonce</i> acak membuat setiap <i>ciphertext</i> unik, namun <i>server</i> belum memvalidasi keunikan <i>nonce</i> |
| <b>Kompleksitas implementasi</b>           | Rendah — hanya menggunakan modul <code>socket</code> bawaan Python  | Sedang — memerlukan library tambahan <code>cryptography</code> dan pemahaman konsep kriptografi                                |
| <b>Overhead performa</b>                   | Minimal                                                             | Sedikit bertambah karena proses enkripsi/dekripsi dan ukuran data yang lebih besar                                             |

|                                        |                             |                                                                  |
|----------------------------------------|-----------------------------|------------------------------------------------------------------|
| <b>Ketergantungan<br/>(dependency)</b> | Tidak ada library eksternal | Memerlukan library<br>cryptography (pip install<br>cryptography) |
|----------------------------------------|-----------------------------|------------------------------------------------------------------|

## 5.2 Perubahan Tingkat Keamanan Berdasarkan Bukti Wireshark

Berdasarkan perbandingan tangkapan Wireshark dari kedua versi aplikasi, perubahan tingkat keamanan sangat signifikan:

1. **Sebelum enkripsi:** Fitur *Follow TCP Stream* pada Wireshark menampilkan seluruh isi percakapan dalam bentuk teks yang dapat dibaca. Pesan seperti "Rapat dosen pukul 10:00 di Ruang Utama" tampil secara jelas dan utuh. Penyerang tidak memerlukan keahlian atau alat khusus selain Wireshark untuk mendapatkan informasi ini.
2. **Sesudah enkripsi:** Fitur *Follow TCP Stream* hanya menampilkan deretan karakter Base64 yang merupakan representasi dari *ciphertext*. Tidak ada bagian dari pesan asli yang terlihat. Bahkan jika penyerang mencoba mendekode Base64 tersebut, yang didapat hanyalah data biner acak karena data tersebut sudah terenkripsi dengan AES-128-GCM.
3. **Perlindungan tambahan:** Jika seseorang mencoba memodifikasi *ciphertext* yang tertangkap lalu mengirimkannya ke *server*, proses dekripsi akan gagal karena *authentication tag* tidak cocok. Ini merupakan lapisan keamanan tambahan yang tidak dimiliki oleh versi tanpa enkripsi.

### 5.3 Risiko yang Masih Tersisa Setelah Enkripsi

Meskipun enkripsi AES-128-GCM telah meningkatkan keamanan secara signifikan, beberapa risiko masih perlu diwaspadai:

1. **Keamanan kunci enkripsi.** Seluruh keamanan sistem bergantung pada kerahasiaan kunci. Jika kunci bocor atau jatuh ke tangan pihak yang tidak berwenang, seluruh pesan yang dienkripsi dengan kunci tersebut dapat didekripsi. Distribusi kunci secara manual juga membawa risiko tersendiri.
2. **Serangan *replay* masih mungkin terjadi.** Meskipun *nonce* acak membuat setiap *ciphertext* unik, *server* dalam implementasi saat ini belum memvalidasi apakah sebuah *nonce* sudah pernah digunakan sebelumnya. Penyerang masih dapat mengirim ulang paket data yang sah.
3. **Tidak ada autentikasi identitas *client*.** Siapa pun yang memiliki kunci dapat mengirim pesan. Tidak ada mekanisme untuk membuktikan identitas spesifik pengirim (misalnya melalui sertifikat digital atau kredensial *login*).
4. **Metadata komunikasi masih terlihat.** Walaupun isi pesan terenkripsi, informasi seperti alamat IP pengirim dan penerima, waktu komunikasi, frekuensi komunikasi, serta ukuran data yang dikirim masih dapat diamati melalui Wireshark. Informasi ini bisa dimanfaatkan untuk analisis pola komunikasi (*traffic analysis*).
5. **Ketergantungan pada satu kunci statis.** Kunci yang sama digunakan untuk seluruh sesi komunikasi tanpa batas waktu. Idealnya, kunci harus dirotasi secara berkala dan setiap sesi sebaiknya menggunakan kunci yang berbeda (*session key*).

## 5.4 Rekomendasi Administrasi Keamanan Jaringan

Berikut adalah enam rekomendasi administrasi keamanan jaringan yang dapat diterapkan untuk meningkatkan keamanan sistem secara keseluruhan:

### 1. Pembatasan Akses Perangkat (*Access Control*)

Terapkan mekanisme kontrol akses pada jaringan LAN untuk memastikan hanya perangkat yang terotorisasi yang dapat terhubung. Langkah konkretnya meliputi:

- Mengaktifkan *MAC address filtering* pada *switch* atau *router*.
- Menerapkan *network segmentation* dengan VLAN untuk memisahkan lalu lintas jaringan berdasarkan departemen atau fungsi.
- Menggunakan *firewall* untuk membatasi akses ke port 12345 hanya dari alamat IP yang terdaftar.

### 2. Pengelolaan Kunci (*Key Management*)

Kunci enkripsi harus dikelola dengan prosedur yang ketat:

- Menggunakan sistem *key management* terpusat untuk mendistribusikan kunci secara aman.
- Menerapkan rotasi kunci secara berkala (misalnya setiap 30 hari) untuk membatasi dampak jika kunci bocor.
- Menghapus kunci lama secara permanen setelah kunci baru diterapkan.
- Mempertimbangkan penggunaan protokol pertukaran kunci seperti Diffie-Hellman untuk sesi mendatang.

### 3. Pencatatan Log (*Logging*)

Setiap aktivitas komunikasi harus dicatat dalam *log* yang mencakup:

- Waktu koneksi, alamat IP, dan port *client*.
- Keberhasilan atau kegagalan proses dekripsi.
- Percobaan koneksi yang ditolak.
- *Log* harus disimpan di lokasi yang aman dan hanya dapat diakses oleh administrator.

#### 4. Validasi *Client* (*Client Authentication*)

Tambahkan mekanisme autentikasi untuk memverifikasi identitas *client* sebelum pesan diproses:

- Menerapkan autentikasi berbasis *token* atau *challenge-response*.
- Menggunakan sertifikat digital (TLS *mutual authentication*) untuk skenario yang memerlukan keamanan lebih tinggi.
- Membatasi jumlah percobaan koneksi gagal untuk mencegah serangan *brute-force*.

#### 5. Pemantauan Koneksi (*Connection Monitoring*)

Lakukan pemantauan secara aktif terhadap koneksi yang masuk ke *server*:

- Menggunakan sistem *Intrusion Detection System* (IDS) untuk mendeteksi pola lalu lintas yang mencurigakan.
- Menerapkan *rate limiting* untuk mencegah serangan *denial-of-service*.
- Membuat notifikasi otomatis jika terjadi anomali seperti lonjakan koneksi dari satu alamat IP.

#### 6. Pembaruan Library (*Dependency Updates*)

Library kriptografi yang digunakan harus selalu dalam versi terbaru:

- Memantau pembaruan keamanan dari library *cryptography* secara berkala.
- Menggunakan alat seperti *pip-audit* untuk memeriksa kerentanan pada dependensi.
- Menguji kompatibilitas sebelum memperbarui library di lingkungan produksi.



## 5.5 Jadwal Pengerjaan Proyek dan Pembagian Tugas

| Tahap | Kegiatan                                      | PIC                    | Durasi |
|-------|-----------------------------------------------|------------------------|--------|
| 1     | Analisis kebutuhan dan perancangan jaringan   | Marsani                | 2 hari |
| 2     | Pembuatan diagram topologi dan flowchart      | Marsani                | 1 hari |
| 3     | Implementasi server dan client tanpa enkripsi | Muhammad Saifulloh     | 2 hari |
| 4     | Pengujian fungsional program                  | Muhammad Saifulloh     | 1 hari |
| 5     | Instalasi Wireshark dan analisis paket        | Kristian Hananiel Hura | 2 hari |
| 6     | Identifikasi kerentanan dan dokumentasi       | Kristian Hananiel Hura | 1 hari |
| 7     | Implementasi AES-128-GCM                      | Sukandar               | 3 hari |
| 8     | Pengujian ulang dengan Wireshark              | Marsani                | 1 hari |
| 9     | Evaluasi komparatif dan rekomendasi           | Sukandar               | 1 hari |
| 10    | Penyusunan laporan akhir dan refleksi         | Seluruh anggota        | 2 hari |

**Timeline keseluruhan:** Pengerjaan dilakukan secara bertahap dan paralel selama kurang lebih dua minggu.

## 5.6 Refleksi Kelompok

### Kendala utama yang dihadapi:

1. *Pemahaman konsep kriptografi.* Beberapa anggota kelompok pada awalnya belum memahami perbedaan antara mode-mode operasi AES (ECB, CBC, GCM) dan mengapa GCM lebih unggul. Kami perlu meluangkan waktu untuk mempelajari konsep *authenticated encryption* sebelum memulai implementasi.
2. *Konfigurasi jaringan untuk pengujian.* Melakukan pengujian pada jaringan LAN yang sebenarnya memerlukan konfigurasi yang tepat, terutama dalam menentukan alamat IP dan memastikan *firewall* tidak memblokir port yang digunakan. Pada beberapa komputer, *Windows Firewall* secara otomatis memblokir koneksi masuk ke port 12345.
3. *Penangkapan paket di localhost.* Saat pengujian dilakukan di satu komputer, paket yang melewati antarmuka *loopback* tidak selalu dapat ditangkap oleh Wireshark tanpa driver tambahan seperti Npcap. Kami harus menginstal Npcap terlebih dahulu sebelum dapat menangkap paket pada antarmuka *loopback*.

### Solusi yang dilakukan:

1. Kami membagi sesi belajar bersama untuk memahami teori kriptografi menggunakan referensi dari buku *Cryptography and Network Security* karya William Stallings. Diskusi kelompok sangat membantu memperjelas konsep-konsep yang abstrak.
2. Untuk masalah konfigurasi jaringan, kami membuat panduan langkah demi langkah untuk menambahkan *exception* pada *Windows Firewall* dan memverifikasi konektivitas menggunakan perintah *ping* sebelum menjalankan program.
3. Masalah Npcap diselesaikan dengan mengunduh dan menginstal versi terbaru dari situs resminya, kemudian memilih opsi untuk mendukung penangkapan paket *loopback*.

### Peningkatan yang dapat diterapkan di masa mendatang:

1. *Pertukaran kunci otomatis.* Menerapkan protokol Diffie-Hellman atau TLS *handshake* agar kunci tidak perlu didistribusikan secara manual.
2. *Dukungan multi-client.* Menggunakan *threading* atau *asyncio* pada Python agar *server* dapat menangani beberapa *client* secara bersamaan.

3. *Antarmuka pengguna grafis (GUI)*. Membangun antarmuka grafis menggunakan *Tkinter* atau *PyQt* agar aplikasi lebih mudah digunakan oleh pengguna non-teknis.
4. *Pencatatan log ke file*. Menambahkan fitur *logging* yang menyimpan seluruh aktivitas komunikasi ke file *log* untuk keperluan audit.
5. *Validasi nonce*. Menambahkan mekanisme penyimpanan *nonce* yang telah digunakan untuk mencegah serangan *replay*.

## Kesimpulan

Proyek ini telah berhasil mendemonstrasikan perbedaan mendasar antara komunikasi jaringan yang tidak terproteksi dan komunikasi yang telah diamankan dengan enkripsi. Melalui pengembangan dua versi aplikasi *client-server* berbasis Python, beberapa hal penting yang dapat disimpulkan antara lain:

1. Komunikasi tanpa enkripsi dalam jaringan LAN sangat rentan terhadap penyadapan. Wireshark membuktikan bahwa pesan yang dikirim dalam bentuk *plaintext* dapat dibaca dengan mudah oleh pihak manapun yang berada di jaringan yang sama.
2. Implementasi AES-128-GCM secara efektif melindungi kerahasiaan dan integritas pesan. Setelah enkripsi diterapkan, data yang tertangkap oleh Wireshark hanya berupa deretan karakter acak yang tidak bermakna.
3. Meskipun enkripsi merupakan langkah krusial, ia bukanlah solusi tunggal yang lengkap. Aspek-aspek seperti pengelolaan kunci, autentikasi *client*, pemantauan jaringan, dan administrasi keamanan lainnya tetap perlu diperhatikan untuk membangun sistem komunikasi yang benar-benar aman.
4. Pemahaman praktis melalui implementasi langsung memberikan pengalaman belajar yang lebih mendalam dibandingkan hanya mempelajari teori. Proses mengidentifikasi kerentanan nyata pada aplikasi yang kita buat sendiri menjadi pengingat kuat akan pentingnya keamanan dalam setiap pengembangan perangkat lunak.

## Daftar Pustaka

1. Stallings, W. (2020). *Cryptography and Network Security: Principles and Practice* (8th ed.). Pearson Education. — Referensi utama mengenai konsep kriptografi simetris (AES), mode operasi cipher block, serta prinsip keamanan jaringan.
2. Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8th ed.). Pearson Education. — Referensi mengenai model OSI Layer, protokol TCP/IP, arsitektur client-server, dan klasifikasi jaringan (PAN, LAN, MAN, WAN).
3. Tanenbaum, A. S., & Wetherall, D. J. (2021). *Computer Networks* (6th ed.). Pearson Education. — Referensi mengenai topologi jaringan, protokol transportasi, dan mekanisme komunikasi data.
4. Dworkin, M. J. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). National Institute of Standards and Technology. — Spesifikasi resmi mode GCM yang digunakan dalam implementasi AES-128-GCM pada proyek ini.
5. Sanders, C. (2017). *Practical Packet Analysis: Using Wireshark to Solve Real-World Network Problems* (3rd ed.). No Starch Press. — Referensi mengenai teknik analisis paket jaringan menggunakan Wireshark, termasuk filter dan interpretasi hasil tangkapan.
6. Paar, C., & Pelzl, J. (2010). *Understanding Cryptography: A Textbook for Students and Practitioners*. Springer-Verlag. — Referensi tambahan mengenai dasar-dasar kriptografi, termasuk penjelasan tentang AES, *key management*, dan *authenticated encryption*.
7. Python Software Foundation. (2024). *socket — Low-level networking interface*. Dokumentasi resmi Python 3.x. Diakses dari <https://docs.python.org/3/library/socket.html> — Dokumentasi modul `socket` yang digunakan dalam implementasi program.
8. Cryptography.io. (2024). *Authenticated encryption with associated data (AEAD)*. Dokumentasi library `cryptography`. Diakses dari <https://cryptography.io/en/latest/hazmat/primitives/aead/> — Dokumentasi library `cryptography` untuk implementasi AES-GCM.